

Integrating Model Transformation in Agent-Oriented Software Engineering

Cuiyun Hu, Xinjun Mao and Hong Ning
Dept. of Computer Science and Technology
National University of Defense Technology, NUDT
Changsha, Hunan Province, China
hcy56316@163.com, mao.xinjun@gmail.com

Abstract—To integrate model transformation technology in agent-oriented software engineering is helpful to improve the quality and efficiency of agent-based software development, and therefore to make AOSE to be widely adopted by researchers and practitioners in software engineering community. This paper integrates model transformation technology into an agent-oriented methodology ODAM that is based on organization metaphor and proposed for complex adaptive systems. The model transformation consists of two steps: from platform-independent models to platform-specific models, from platform-specific models to code. The transformation algorithms in each phase are designed. A software tool that implements the algorithms has been developed as a part of agent-oriented supporting environment.

Keywords—model transformation; agent-oriented software engineering; model-driven development

I. INTRODUCTION

Agent-oriented software engineering (AOSE) is considered to be a new and powerful paradigm to develop complex systems [1]. In current practice, a multi-agent system is typically designed agent-orientation by using an agent-oriented methodology, but is implemented in traditional programming languages (i.e., OO programming language) [2]. This leaves a great gap between design and implementation, which hampers the wide acceptances and adoptions of agent technology in industry.

In the past years, there are many software engineering technologies that are proved to be effective to deal with complexity and improve the development quality and efficiency. To integrate AOSE with these technologies is important to make AOSE more acceptable and usable and to enrich the potentials of AOSE to support the development of complex systems. Model Driven Architecture (MDA) proposed by OMG advocates taking models as the main artifacts of software engineering and software development as a series of model transformations [3]. Reuse of models and mappings between models effectively improve the quality and efficiency of software development, which has been proved in OO software engineering.

This paper proposes an approach to integrating model transformation technology in AOSE, in order to bridge the gap between design and implementation. The idea is specified in an agent-oriented methodology ODAM [4], which is based on organization metaphor and supporting complex adaptive MAS. The core of our work is to investigate the model transformation algorithms. The rest of this paper is structured as follows. Section 2 discusses the role of model transformations in ODAM. Section 3 introduces the model transformation algorithms in details. Section 4 describes supported toolkits that implement the algorithms. Conclusions are made in Section 5.

II. MODEL TRANSFORMATIONS IN ODAM

ODAM supports the complex MAS development in four major phases: requirement, analysis, architecture design and implementation. For the first three phases, ODAM represents the MAS with five types of platform-independent models (PIM) specified by organization meta-concepts. They are: *Organization sCenario Model (OCM)*, which describes the initial function requirements of MAS in requirement phase; *Role Behavior Model (RBM)*, which defines the behaviors of each role in analysis phase, including responsibilities, activities, services, environment and resources; *Organization Interaction Model (OIM)*, which defines how agents in organization interact with each other to achieve organization scenarios in analysis phase. *Role Transition Model (RTM)*, which defines the dynamic and adaptation property of self-adaptive agents in analysis phase; *Organization Structure Model (OSM)*, which defines the overall structure of MAS in architectural design phase. These models are used to specify and analyze complex MAS from behavior and structure viewpoints in different abstract levels, and a detail description of them has been give in [4].

In the implementation phase, the agent-oriented development platform SADE (Self-Adaptation Development Environment) is used to program and deploy MAS. In SADE, an agent-oriented system is composed by a group of self-adaptive agents and a number of roles which encapsulate the behaviors and environment of the

agents. An agent can play multiple roles and the agent can change its roles according to its situated environment using the adaptation logics defined by SADL (Self-Adaptation Description Language). Therefore, three types of models are designed to construct adaptive MAS.

Role Model (RM), defines the elementary modular units of software system, including internal events, attributes, behaviors and environments. The environments of a role are specified in the terms of type, name, their properties and constraints.

Adaptive Agent Model (AAM), defines the elementary execution unit, i.e., adaptive agents of the software system. The dynamic binding mechanism is specified in it.

Adaptive Strategy Model (ASM), defines the self-adaptation of agents. Each self-adaptation program consists of a number of self-adaptive strategies. Each self-adaptive strategy is composed of a number of adaptive rules that define how agent adapts to the changes of environment and its internal state based on the dynamic binding mechanism. Adaptive rules has the following syntax format: “WHEN (*ChangeExp*) IF (*StateExp*){(*AdaptiveAction()*)+ }”, which means that when the specified internal or environment change *ChangeExp* occurs, and if agent has specified state *StateExp*, then agent should take adaptive actions {(*AdaptiveAction()*)+} to adjust its structures and behaviors. These models are platform-specific models (PSM) (i.e., relying on the specific implementation technology, software architecture and programming language) and can be obtained automatically from the PIMs in the first three phases with model transformation technology.

III. MODEL TRANSFORMATION ALGORITHMS

The model transformations in ODAM are performed in two steps: from PIMs to PSMs, from PSMs to implementation codes. The transformation algorithms in each step are designed following.

A. Algorithm Transforming PIMs to PSMs

Fig. 1 defines the transforming algorithm to map PIMs to PSMs. The inputs of the algorithm are PIMs, including organization structure model, organization interaction model, role behavior model and role transition model. The outputs of the algorithm are PSMs, including role model, adaptive agent model and adaptive strategy model.

PIM2PSM algorithm consists of a number of steps. Firstly, the role model is generated based on the organization structure model. In our approach, the role model and the role in organization structure model have the one-to-one relationship. Second, each generated role model should be further refined in order to incorporate more design information based on other related models, such as role behavior model, role interaction model, etc. According to our implementation architecture, each role has its properties and encapsulates its environment, behaviors, etc. Therefore, the constituents of role model should be

elaborated. Third, the agent mode is generated based on the organization interaction model and role transition model. Typically, if a number of agents have the same adaptation characteristics, then a corresponding agent mode should be generated for them. Lastly, the adaptive strategy model for each adaptive agent model should be generated.

```

Algorithm PIM2PSM (OSM, OTM, RBM, OIM, RTM):
RM, AAM, ASM
// Generating Role Models
For (each role r in OSM)
    Generate a role model RMr;
    RMr.name = r.name;
    If (role r inherits from another role r')
        Generate a role model RMr';
        RMr'.name = r'.name;
        Set the parent of RMr as RMr';
    End If
End For
// Refining Role Models
For (each RBM and the role r in it)
    Find the role model RMr generated from r;
    //Generate behaviors of RM
    For (each responsibility rp in RBM)
        Generate a behavior bev of RMr;
        bev.name = rp.name;
        If (rp is decomposed as sub-reponsibilities)
            For (each sub-reponsibility srp of rp)
                Generate sub-behavior sbev of srp;
                sbev.name = srp.name;
            End For
        End If
    End For
End For
// Generate environments of RM
For (each environment env in RBM)
    Generate an environment env of RMr;
    RMr.env.name = env.name;
End For
//Generate attributes of RM
For (each resource res in RBM)
    Generate a Class c;
    c.name = res.name;
    Generate an attribute attr of RMr;
    attr.type = c.name;
End For
End For
//Generate AAM and ASM
For (each RTM that describes agent agt)
    Generate a AAM;
    AAM.name = agt.name;
    Generate ASM for AAM;
    ASM.name = agt.name + "Strategy";
    // Generate adaptive rules of ASM
    For (each transition tran in RTM)
        Generate an adaptive rule rl of ASM;
        Set change event of rl as the event of tran;
        Set action set of rl as action set of tran;
        For (each active r in source configuration of tran)
            Generate a state st of rl;
            st.name = r.name + "@Active";
        End For
        For (each inactive r in source config. of tran)
            Generate a state st of rl;
            st.name = r.name + "@Inactive";
        End For
    End For
End For
End For

```

Figure 1. Algorithm for transforming PIMs to PSMs

In our approach, the output models of PIM2PSM algorithm are specified by XML and should be further transformed in order to get the ultimate program code framework.

B. Algorithm for Transforming Platform-Specific Models to Implementation Codes

SADE provides a reusable software package written in Java to support the development of role model and agent model, and a language SADL to describe the adaptive strategy of adaptive agent. Especially, the reusable software package provides an abstract classes Role that encapsulates the internal events, attributes, behaviors and environments and an abstract class SAgent that encapsulates the dynamic binding mechanism. Therefore, based on the role models, agent models and strategy models, there are three kinds of code frameworks that should be transformed: role classes, agent classes and adaptive strategies. Fig. 2 defines the transformation algorithm to map the PSMs to program code framework.

```

Algorithm PSM2CODE (RM, AAM, ASM): Code
// Generating Role Classes
For (each RM rm)
    Generate a role class rc inheriting Role;
    rc.name = rm.name;
    For (each attribute attr in rm)
        Generate the attr of rc.
    End For
    For (each behavior bev in rm)
        Generate behavior class bevc inheriting Behavior;
        bevc.name = bev.name;
        Generate methods “action()”, “done()” of bevc;
    End For
    Generate and set “getEnvironments()” of rc;
    For (each service serv in rm)
        Generate ServiceDescription of serv in “Init()” of rc;
        Generate service registration code of serv in “Init()”;
        Generate service unregistration code of serv in “exit()” of rc;
    End For
End For
// Generating Agent Classes
For (each AM am)
    Generate an agent class ac inheriting SAgent;
    ac.name = am.name;
    Generate “setup()” and “takeDown()” methods of ac;
    Set the strategy of the agent in “setup()”;
End For
// Generating Adaptive Strategy Code
For (each AS as)
    Generate the adaptive strategy code based on the template;
End For

```

Figure 2. Algorithm transforming PSM to code

In our approach, we adopt the template-based approach to implement code generation by using JET (Java Emitter Templates) [5]. As a part of EMF (Eclipse Modeling Framework), JET provides a JSP-like syntax to write the code template and is a generic template engine to generate like SQL, XML, Java source code and other output from templates. According to the syntax definition of role class,

agent class and SADL in SADE, we design three code templates respectively. Fig. 3 depicts the adaptive strategy template.

```

<%@ jet package="edu.nudt.sade.codegen.templates"
class="StrategyTemplate"
imports="java.util.* edu.nudt.sade.core.implement.*"%>
<%
    SStrategy strategy = (SStrategy)argument;
    SAgent agent = strategy.getOwner();
    String packageName = agent.getPack().getName();
    String strategyName = strategy.getName();
    List<SRule> rules = strategy.getRules();
%>
package <%=packageName>".strategies"%>;
<%int loc = packageName.lastIndexOf(".");%>
import <%=packageName.substring(0,loc)%>.*;
use <%=packageName%>.<%=agent.getName()%>;
program <%=strategyName%>{
    InitStrategy {
        join(self.getInitRole());
    }
<%if(rules!=null&&!rules.isEmpty())%>
    strategy AdaptiveStrategy{
        <%for(int i=0;i<rules.size();i++)%>
            SRule rule = rules.get(i);
            String event = rule.getEvent();
            List<String> states = rule.getStates();
            List<String> actions = rule.getActions();
            String state="";
            for(int j=0;j<states.size();j++){
                if(state.equals("")){ state = states.get(j);
                }
                else{ state = state+"&" + states.get(j);
                }
            }
            when(/*<%=event%>*/) {
                <%if(!state.equals(""))%>
                if(<%=state%>){
                    <%if(actions!=null&&!actions.isEmpty())%>
                    for(int j=0;j<actions.size();j++)%> <%=actions.get(j)%>;
                    <%
                    }%>
                }%>
                <%else%>
                <%if(actions!=null&&!actions.isEmpty())%>
                for(int j=0;j<actions.size();j++)%> <%=actions.get(j)%>;
                <%
                }%>
            }
        }%>
    }%>
}
<%}%>

```

Figure 3. Template of the strategy code

IV. ALGORITHMS IMPLEMENTATION

We have developed a software tool to implement the above algorithms and support the model transformations in an automatic way. The software tool is one element of the whole toolkit that consists of three software tools: visual modeling tool, model checking tool and model transformation tool. Visual modeling tool and model checking tool support the modeling and checking the consistency of adaptive MAS, and for more details see [4][7]. Model transformation tool, based on the established

models, is responsible to transform the organization-based models into platform-specific models and generate the implementation codes framework. Fig. 4 depicts the technical framework.

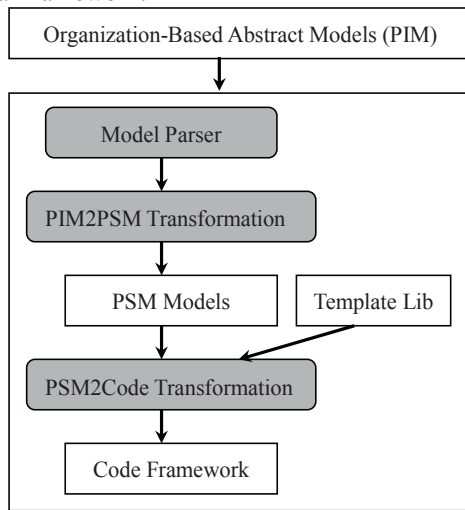


Figure 4. Technical framework of software tool

V. CONCLUSIONS

Obviously, to integrate effective and successful software engineering technology (such as software reuse, model-driven development, iteration development) with agent technology is important for AOSE to be widely accepted by practitioners and used in various domains. One of the contributions in this area is the model-driven architecture and software development [3]. Recently, such technology is notably introduced into the literature of AOSE and merged with software agent technology to support the development of MAS [7]. Some of researchers focus on the model transformation to bridge the gap between design and implementation in the process of MAS development [8][9][10], and others focus on the reusability to generate implementation models of various platforms [7][11].

In this paper, we have presented an approach to integrate model transformations and code generations with AOSE in the scope of ODAM methodology. In this study, the model transformation algorithms were designed and the software tool that implements the algorithms and supports the model transformation in an automatic way has been developed. Moreover, the supported software tool is integrated with the modeling software tool and development platform.

There are still some drawbacks in our existing approach. Firstly, the generated code framework is coarse-grain.

More detailed design of adaptive systems should be performed by software designers. Second, our algorithms and corresponding software tool only support to generate the software structure and program codes that satisfy the functional requirements defined in the organization-based models. Non-functional requirements and design optimization of the software systems are not considered in our approach. Third, the interaction and coordination between agents are not considered in our algorithms. In future work, such work should be incorporated.

ACKNOWLEDGMENT

The authors gratefully acknowledge the financial support from Natural Science Foundation of China under granted number 60773018, High-Tech 863 Program of China under granted number 2007AA01Z135.

REFERENCES

- [1] N.R.Jennings, "An Agent-based Approach for Building Complex Software Systems," *Communication of ACM*, vol. 44, no. 4, 2001, pp. 35-41.
- [2] Rafael H. Bordini, Mehdi Dastani, and Michael Winikoff, "Current Issues in Multi-Agent Systems Development," *Post-proceedings of the Seventh Annual International Workshop on Engineering Societies in the Agents World, LNAI 4457*, 2007, pp. 38-61.
- [3] S. J. Mellor, K. Scott, A. Uhl, and D. Weise, *MDA Distilled*, Addison-Wesley, 2004.
- [4] Xinjun Mao, Ji Wang, "Engineering Adaptive Multi-agent Systems with ODAM Methodology," *Proceedings of Prima 2007, LNAI*, Springer, 2009, pp.380-385.
- [5] Java Emitter Templates Tutorial, http://eclipse.org/articles/ArticleJET2/jet_tutorial2.html
- [6] Xinjun Mao, Hong Ning, Cuiyun Hu, and Ji Wang. Modeling and Checking Adaptive MAS with ODAMTools. *Communications of SIWN (CoSIWN)*, vol. 3, no. 1, 2008, pp. 53-57.
- [7] Juan Pavón, Jorge J. Gómez Sanz, and Ruben Fuentesl, "Model Driven Development of Multi-Agent Systems," *Proceeding of The 2nd European Conference on Model Driven Architecture, LNCS*, Springer Verlag, 2006, pp.284-298.
- [8] L. Penserini, A. Perini, A. Susi, and J. Mylopoulos., "From Stakeholder Intentions to Software Agent Implementations," *Proceedings of the 18th Conference On Advanced Information Systems Engineering, LNCS*, Springer-Verlag, vol. 4001, 2006, pp. 465-479.
- [9] Perini and A. Susi, "Automating Model Transformations in Agent-Oriented Modeling," *Agent Oriented Software Engineering VI (AOSE'05)*, LNCS, Springer-Verlag, 2006, pp. 167-178.
- [10] Amor M., Fuentes L., and Vallecillo A., "Bridging the Gap between Agent-Oriented Design and Implementation Using MDA," *Agent-Oriented Software Engineering V (AOSE 2004)*, *Lecture Notes in Computer Science*, Springer-Verlag, vol. 3382, 2005, pp. 93-108.
- [11] Cossentino, M., "From Requirements to Code with the PASSI Methodology," *Agent-Oriented Methodologies*, Idea Group Pub, Henderson-Sellers, 2005, pp. 79-106.